

تقرير عملي

تحليل النصوص باستخدام Hadoop MapReduce

تطبيق WordCount على بيئة Cloudera QuickStart VM داخل VMware

يشمل HDFS و MapReduce و YARN واستخراج النتائج وتحليلها

اسم الطالب: أحمد أنور عطران

المستوى الدراسي: المستوى الرابع

الجامعة: الجامعة الماليزية

اسم المقرر: بيانات ضخمة (Big Data Analytics)

أستاذ المادة: د. رمزي

تاريخ التسليم: 2026

1. الملخص التنفيذي

أُعد هذا التقرير للطالب **أحمد أنور عطران** من المستوى الرابع في **الجامعة الماليزية**، ضمن مقرر **بيانات ضخمة (Big Data Analytics)** بإشراف **د. رمزي**، على أن يكون تاريخ التسليم **2026**.

يشرح هذا التقرير دورة تحليل نصوص باستخدام منظومة Hadoop. تبدأ الدورة بالحصول على ملف نصي وفحصه في نظام Linux، ثم إنشاء مساحة عمل داخل HDFS ورفع البيانات إليها. بعد ذلك تُشغّل مهمة WordCount بواسطة MapReduce، بينما يتولى YARN إدارة موارد التنفيذ. تُقرأ النتائج من HDFS وتُرتَّب أو تُبحث باستخدام أدوات Linux.

النتيجة الأساسية: تُظهر لقطات الشاشة تنفيذًا فعليًا لمهمة WordCount على عدة كتب داخل المسار `/bigdata/books`. أما الملف النصي المرفق فيشرح المسار التعليمي نفسه باستخدام كتاب *The Adventures of Sherlock Holmes* داخل `/bigdata/sherlock`.

2. نطاق التقرير ومصدر المعلومات

اعتمد التقرير على مصدرين مرفقين: ملف شرح تفصيلي لمشروع Hadoop MapReduce، وصورتين من شاشة بيئة Cloudera. لذلك جرى التمييز بين الأوامر المقترحة في الشرح وبين الأوامر والنتائج التي يمكن قراءتها مباشرة من الصور.

المصدر	ما يثبته
الملف النصي المرفق	المنهج الكامل للمشروع، الأهداف، الأوامر، شرح HDFS و MapReduce و YARN، ومراحل استخراج النتائج.
الصورة الأولى	عرض ملفات ناتج WordCount داخل <code>/bigdata/books/output</code> وقراءة <code>part-r-000000</code> ثم ترتيب النتائج تنازليًا.
الصورة الثانية	البحث عن كلمة London، دمج مخرجات HDFS في ملف محلي، ثم إنشاء ملف نتائج مرتبة.

3. فكرة المشروع ودورة البيانات

الهدف من WordCount هو حساب عدد مرات ظهور كل كلمة في النص. يقوم Mapper بإنتاج أزواج من الشكل `word, 1`، ثم يجمع Shuffle القيم المتشابهة، ويحسب Reducer المجموع النهائي لكل كلمة.

REAL TEXT → Linux → HDFS → Mapper → Shuffle → Reducer → Output in HDFS → Linux Analysis

في الشرح التعليمي يظهر المسار باسم `/bigdata/sherlock` لكتاب Sherlock Holmes. وفي الصور الفعلية يظهر المسار باسم `/bigdata/books`، وهو تطبيق موسع على عدة كتب.

4. خطوات التنفيذ المعيارية

4.1 الحصول على البيانات وفحصها

يقترح الشرح تنزيل كتاب *The Adventures of Sherlock Holmes* من Project Gutenberg باسم `1661-0.txt` ، ثم التحقق من الملف باستخدام أوامر Linux. تُستخدم أوامر مثل `ls -lh` لمعرفة الحجم، و `head` و `tail` لفحص المحتوى، و `wc -l` و `wc -w` لعد الأسطر والكلمات.

```
cd /home/cloudera
wget https://www.gutenberg.org/files/1661/1661-0.txt
ls -lh 1661-0.txt
head -20 1661-0.txt
wc -l 1661-0.txt
wc -w 1661-0.txt
```

4.2 إنشاء مجلدات HDFS ورفع الملف

```
hdfs dfs -mkdir -p /bigdata/sherlock/input
hdfs dfs -put 1661-0.txt /bigdata/sherlock/input/
hdfs dfs -ls -h /bigdata/sherlock/input
hdfs dfs -cat /bigdata/sherlock/input/1661-0.txt | head -20
hdfs dfs -du -h /bigdata/sherlock/input
```

4.3 تشغيل WordCount

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar wordcount \
/bigdata/sherlock/input \
/bigdata/sherlock/output
```

في حالة التطبيق متعدد الكتب، يُستبدل مسار الإدخال والإخراج بالمسارات الفعلية المستخدمة، مثل `/bigdata/books/input` و `/bigdata/books/output`.

4.4 قراءة النتائج وتحليلها

```
hdfs dfs -ls /bigdata/sherlock/output
hdfs dfs -cat /bigdata/sherlock/output/part-r-000000 | head -20
hdfs dfs -cat /bigdata/sherlock/output/part-r-000000 | sort -k2 -nr | head -20
hdfs dfs -getmerge /bigdata/sherlock/output sherlock_wordcount.txt
```

الملف `_SUCCESS` مؤشر على انتهاء المهمة بنجاح، وليس ملف النتائج التحليلية. النتائج الفعلية تكون في ملف أو ملفات من نوع `.part-r-*`.

5. ما يظهر في لقطات الشاشة

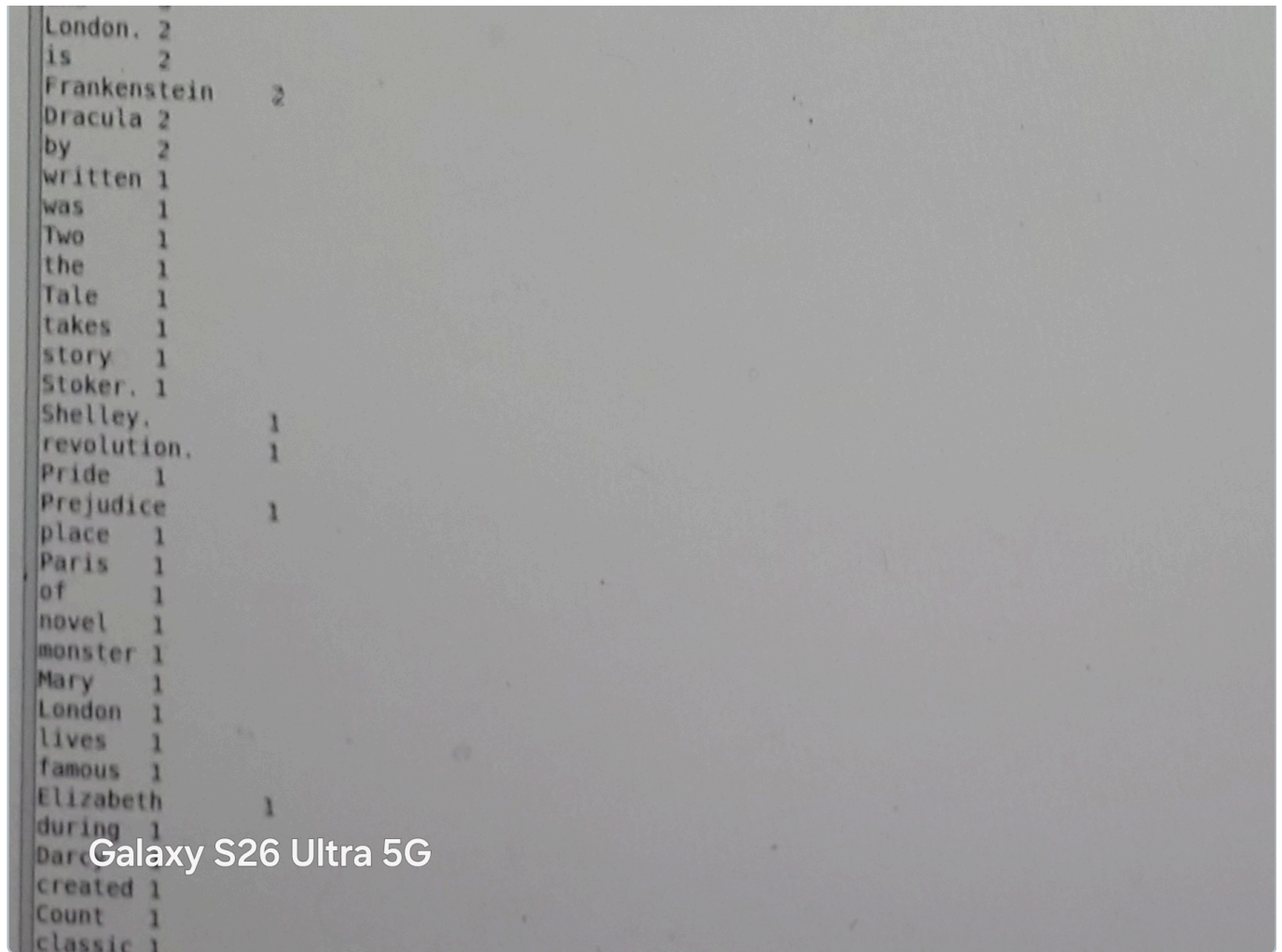
5.1 التحقق من مخرجات HDFS

تُظهر الصورة الأولى تنفيذ الأمر `hdfs dfs -ls /bigdata/books/output` ، مع ظهور عنصرين: `_SUCCESS` و `part-r-000000`. ويظهر حجم ملف النتائج قريبًا من 333 بايت، وفق القراءة الظاهرة في الشاشة.

```

File Edit View Search Terminal Help
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters:
  Bytes Read=311
File Output Format Counters:
  Bytes Written=333
[cloudera@quickstart ~]$ hdfs dfs -ls /bigdata/books/output
Found 2 items
-rw-r--r-- 1 cloudera supergroup 0 2026-09-23 15:47 /bigdata/books/output/ SUCCESS
-rw-r--r-- 1 cloudera supergroup 333 2026-09-23 15:47 /bigdata/books/output/part-r-00000
[cloudera@quickstart ~]$ hdfs dfs -cat /bigdata/books/output/part-r-00000
A 1
Bram 1
Cities 1
Count 1
Darcy 1
Dracula 2
Elizabeth 1
Frankenstein 2
London 1
London. 2
Mary 1
Paris 1
Prejudice 1
Pride 1
Shelley. 1
Stoker. 1
Tale 1
Two 1
a 4
about 1
and 3
by 2
castle. 1
classic 1
created 1
during 1
famous 1
in 4
is 2
lives 1
monster 1
novel 1
of 1
place 1
revolution. 1
story 1
takes 1
the 1
was 1
written 1
[cloudera@quickstart ~]$ hdfs dfs -cat /bigdata/books/output/part-r-00000 | sort
-k2 -nr
in 4
a 4
and 3

```



الصورة الأولى: قراءة part-r-00000 ثم ترتيب النتائج باستخدام sort.

5.2 قراءة الكلمات وترتيبها

من النص الظاهر في الصورة يمكن قراءة أمثلة من ناتج العد، منها: London 1 ، London 1 ، Frankenstein 2 ، Dracula 2 ، in 4 ، and 3 ، a 4 ، و 2 by . كما يظهر أن الترتيب التنازلي يعرض كلمات مثل In 4 و a 4 في البداية.

ملاحظة تحليلية: يعامل WordCount الأساسي الكلمات ذات اختلاف حالة الأحرف أو علامات الترقيم كقيم مختلفة؛ لذلك ظهرت London و London في سطرين منفصلين. وهذا يوضح ضرورة تنظيف البيانات قبل التحليل المتقدم.

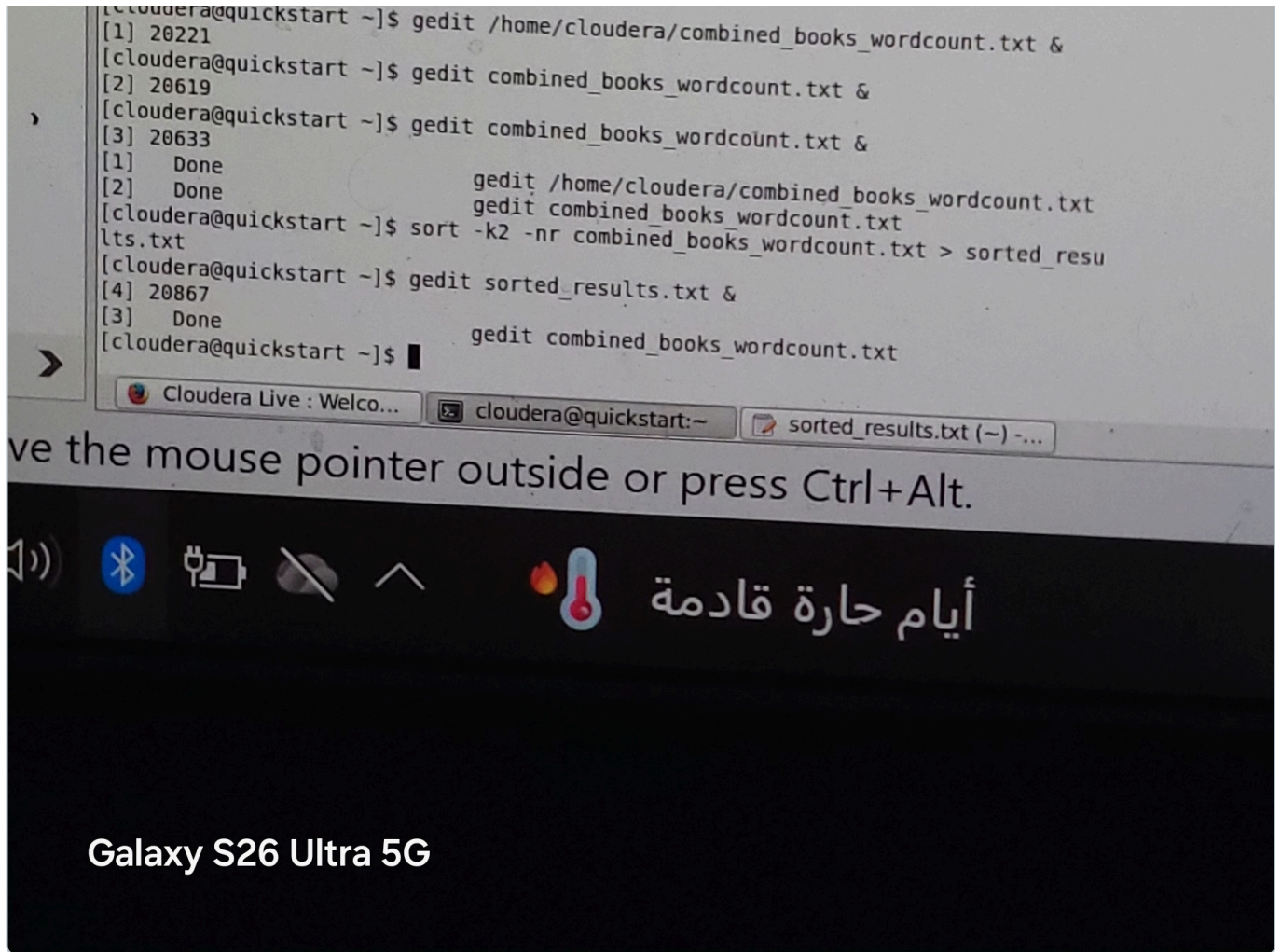
5.3 استخراج الناتج والبحث داخله

تُظهر الصورة الثانية استخدام `grep -w "London"` للبحث عن الكلمة، ثم تنفيذ `hdfs dfs -getmerge` لإنشاء ملف محلي. بعد ذلك استُخدم الأمر `sort -nr` لحفظ ترتيب النتائج في الملف `sorted_results.txt`.

```

Paris 1
of 1
novel 1
monster 1
Mary 1
London 1
lives 1
famous 1
Elizabeth 1
during 1
Darcy 1
created 1
Count 1
classic 1
Cities 1
castle. 1
Bram 1
about 1
A 1
[cloudera@quickstart ~]$ hdfs dfs -cat /bigdata/books/output/part-r-00000 | grep
-w "London"
London 1
London. 2
[cloudera@quickstart ~]$ hdfs dfs -getmerge /bigdata/books/output combined_books
_wordcount.txt
[cloudera@quickstart ~]$ ls -lh combined_books_wordcount.txt
-rw-r--r-- 1 cloudera cloudera 333 Sep 23 15:51 combined_books_wordcount.txt
[cloudera@quickstart ~]$ cat combined_books_wordcount.txt
A 1
Bram 1
Cities 1
Count 1
Darcy 1
Dracula 2
Elizabeth 1
Frankenstein 2
London 1
London. 2
Mary 1
Paris 1
Prejudice 1
Pride 1
Shelley. 1
Stoker. 1
Tale 1
Two 1
a 4
about 1
and 3
by 2
castle. 1
classic 1
created 1
during 1
famous 1
in 4
is 2
lives 1
monster 1
novel 1
of 1
place 1
revolution. 1
story 1
takes 1
the 1
was 1
written 1

```



الصورة الثانية: البحث عن London، استخدام getmerge، ثم إنشاء sorted_results.txt.

6. تفسير مكونات Hadoop في التطبيق

المكوّن	وظيفته في المشروع
Linux	تنزيل البيانات وفحصها واستقبال النتائج بعد استخراجها من HDFS.
HDFS	تخزين ملفات الإدخال ومخرجات WordCount بطريقة موزعة داخل مسارات منظمة.
Mapper	تحويل كل كلمة إلى زوج أولي من الشكل 1, word.
Shuffle	تجميع القيم التي تحمل الكلمة نفسها قبل عملية الجمع.
Reducer	جمع القيم وإنتاج العدد النهائي لكل كلمة.
YARN	إدارة موارد تشغيل المهمة ومتابعة حالتها من خلال واجهة ResourceManager.
getmerge	جمع ملفات مخرجات HDFS في ملف واحد على نظام Linux.

7. إجابات الأسئلة التحليلية

أين توجد النتيجة النهائية؟ توجد أولاً داخل HDFS في ملفات `part-r-*` ضمن مجلد الإخراج. وبعد استخدام `getmerge` تصبح نسخة محلية قابلة للقراءة والتحليل.

لماذا قد تختلف النتائج عن التوقع؟ لأن `WordCount` الجاهز لا يؤخذ حالة الأحرف ولا يزيل علامات الترقيم. لذلك لا ينبغي جمع `London` و `London .` يدويًا إلا بعد تطبيق سياسة تنظيف واضحة.

ما الفرق بين Linux وHDFS؟ ملف `Linux` موجود في نظام الملفات المحلي للجهاز، بينما ملف `HDFS` موجود داخل نظام التخزين الموزع الخاص بـ `Hadoop`. الأمر `put` يرفع نسخة إلى `HDFS`، ولا يعني أن الملف المحلي اختفى.

ما وظيفة YARN؟ `YARN` لا يحسب الكلمات بنفسه؛ بل يدير موارد تشغيل المهمة ويتابع تطبيق `MapReduce` وحالتي `Map` و `Reduce`.

8. تقييم التنفيذ الموثق

الصور تثبت نجاح مرحلة إخراج `WordCount`، ووجود ملف نتائج قابل للقراءة، والقدرة على البحث داخل النتائج، ونسخها إلى `Linux`، ثم ترتيبها. ولا تظهر في الصور المرفقة واجهة `YARN` نفسها، كما لا تظهر القيم الخاصة بـ `Holmes` و `Watson` و `Sherlock` و `Lestrade`؛ لذلك لم تُدرج أرقام غير موثقة لهذه الكلمات.

لإكمال التقييم التجريبي: بعد تنفيذ نسخة `Sherlock` يمكن استخراج الكلمات المطلوبة بالأوامر التالية ثم تسجيل القيم في جدول النتائج.

```
for word in Holmes Watson Sherlock Lestrade London; do
  hdfs dfs -cat /bigdata/sherlock/output/part-r-00000 | grep -w "$word"
done
```

9. تحسينات مقترحة

1. توحيد الأحرف إلى lowercase قبل العد.
2. إزالة علامات الترقيم من الكلمات.
3. استبعاد الكلمات الشائعة إذا كان الهدف تحليل الموضوع أو الشخصيات.
4. حفظ سجل تنفيذ المهمة ولقطة من واجهة `YARN` لتوثيق إدارة الموارد.
5. تكرار التجربة على عدة ملفات ومقارنة النتائج بين الكتب بدل دمجها فقط.

10. الخلاصة

يوضح المشروع دورة عمل متكاملة تبدأ من بيانات حقيقية وتنتهي بنتائج قابلة للتحليل. يبين التنفيذ أن `HDFS` مسؤول عن التخزين، وأن `MapReduce` مسؤول عن المعالجة الموزعة، وأن `YARN` يدير موارد التشغيل، بينما تُستخدم أدوات `Linux` مثل `grep` و `sort`

لتحليل المخرجات بعد إنتاجها. كما تؤكد النتائج المصورة أهمية تنظيف النص قبل تفسير الأعداد، لأن اختلاف حالة الأحرف وعلامات الترقيم قد يقسم الكلمة الواحدة إلى أكثر من قيمة.

المراجع

1. ملف شرح مشروع Hadoop MapReduce المرفق من المستخدم.
2. Project Gutenberg — The Adventures of Sherlock Holmes.

أُعد هذا التقرير اعتمادًا على الملفات المرفقة. الأرقام والأوامر المنسوبة إلى لقطات الشاشة مقتبسة مما يظهر فيها، بينما الأوامر الأخرى واردة في الشرح النصي المرفق.